// DOCUMENT QUESTION AND ANSWER

RAG: give a model your documents

How a program answers from your own files, explained with models you can click



// 08 RAG EXPLAINED

A model cannot know your documents

QUESTION **What payment terms do servo motor suppliers get at Aldermoor?**

Without documents | With the policy page

WHAT THE MODEL RECEIVES

What payment terms do servo motor suppliers get at Aldermoor?

WHAT IT ANSWERS

Generic advice, no company facts

Public information specifically detailing procurement or payment terms for servo motor suppliers at an entity named Aldermoor is not publicly available.

Depending on the context, here is how this typically breaks down:

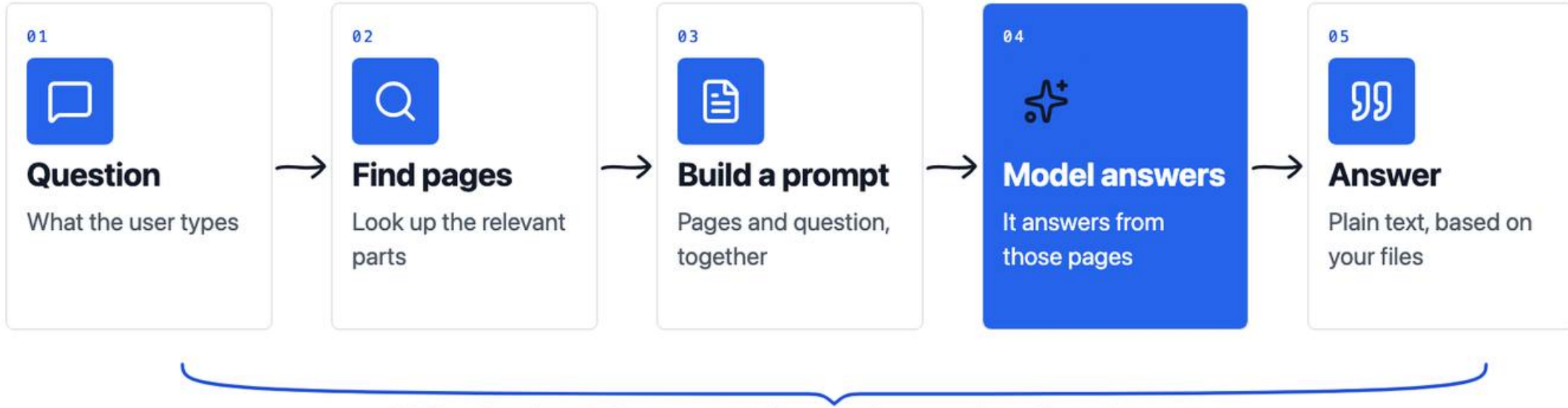
1. Private Commercial Contracts: If Aldermoor refers to a private manufacturing company, OEM, or systems integrator, specific payment terms (e.g., Net 30, Net 60, stage payments, or letters of credit) are negotiated privately under commercial non-disclosure or purchase order terms and conditions.
2. Industry Standards: In industrial ...

Observed with gemini-3.8-flash on 10 October 2026, 11.0 s. The wording changes between runs; the answer never contains Aldermoor's real rule.

Click both buttons. The second answer comes from the page that was placed in the prompt, not from the model's memory.

// 08 RAG EXPLAINED

The idea: look it up, then answer



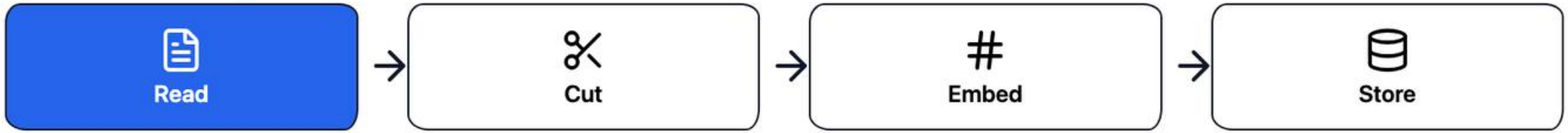
RAG = Retrieval-Augmented Generation: retrieve first, then generate

like an open-book exam: find the page, then write the answer

// 08 RAG EXPLAINED

Two stages: before and during a question

Before questions (indexing) | At question time (asking)



1. Read each PDF page

PyPDFLoader turns every page of the PDF into text and remembers where it came from (the file name).

```
rag/data_ingestion.py
PyPDFLoader(local_path).load()
```

Back | Next step | Play

Before questions (indexing): step 1 of 4

// 08 RAG EXPLAINED

Cut documents into chunks

STRATEGY Recursive (LangChain) CHUNK SIZE 300 OVERLAP 40 Meridian setting: 1000 / 100 Edit text

CHUNKS	AVERAGE SIZE	LARGEST	OVERLAPPING CHARS	AVERAGE OVERLAP
14	232	297	148	11

SOURCE TEXT (HOVER A COLOUR TO SEE ITS CHUNK)

Aldermoor Industries - IFRS Accounting Policy
(fictional)
Capitalisation
Assets with a useful life of more than one year and a cost above EUR 5,000 are capitalised as property, plant and equipment under IAS 16. Smaller items and consumables are expensed immediately.
Depreciation schedule
Filling line equipment is depreciated on a straight-line basis over 10 years. Production robots and
tooling are depreciated over 8 years. IT hardware and servers are depreciated over 4 years. Buildings are depreciated over 30 years.
General ledger ranges
Fixed assets use accounts 0xxx. Material costs use 5xxx. External services use 6xxx. Manufacturing overhead uses 4xxx.

Aldermoor Industries - Annual Report FY2024

CHUNKS THE RETRIEVER WILL STORE

CHUNK 1 · 289 CHARS
Aldermoor Industries - IFRS Accounting Policy (fictional) Capitalisation Assets with a useful life of more than one year and a cost above EUR 5,000 are capitalised as property, plant and equipment un …

CHUNK 2 · 289 CHARS
immediately. Depreciation schedule Filling line equipment is depreciated on a straight-line basis over 10 years. Production robots and tooling are depreciated over 8 years. IT hardware and servers are …

CHUNK 3 · 140 CHARS
General ledger ranges Fixed assets use accounts 0xxx. Material costs use 5xxx. External services use 6xxx. Manufacturing overhead uses 4xxx.

CHUNK 4 · 297 CHARS
Aldermoor Industries - Annual Report FY2024 (fictional) Company overview Aldermoor Industries is a fictional German manufacturer of industrial packaging and filling machinery, headquartered in Hambur …

CHUNK 5 · 284 CHARS
Financial highlights Total revenue in FY2024 was EUR 2.84 billion, up from EUR 2.61 billion in FY2023 (an increase of 8.8 percent). Revenue in FY2022 was EUR 2.37 billion. EBITDA in FY2024 was EUR 398 …

The recursive splitter is a port of LangChain RecursiveCharacterTextSplitter: it tries to cut at paragraph breaks first, then line breaks, then spaces. Smaller chunks match precisely but lose context; larger chunks keep context but blur the match.

// 08 RAG EXPLAINED

Embeddings: meaning as numbers

Text goes in, 768 numbers come out

The list of numbers is called an embedding. It describes meaning, not spelling.

Similar meaning, similar numbers

Two sentences about payment terms get numbers that point in nearly the same direction.

Closeness is a number

The usual measure is cosine similarity: 1 is identical direction, 0 is unrelated.

a tiny example, then a real one

```
# three numbers instead of 768
"Servo motors are paid on Net-45"   [0.9, 0.1, 0.0]
"Filling line equipment, 10 years"   [0.1, 0.9, 0.1]
"When do we pay servo suppliers?"   [0.8, 0.2, 0.1]
^ the question is closest to the first text

# real numbers, gemini-embedding-001, run on 10 October 2026
0.85  Servo motors are paid on Net-45 terms
0.55  Filling line equipment is depreciated over 10 years
0.57  Revenue in FY2024 was EUR 2.84 billion
```

The real scores come from try_embeddings.py, which we run in the next section.

// 08 RAG EXPLAINED

Keyword search misses what meaning search finds

PICK A QUESTION

When do we pay servo motor suppliers? | Who has to approve a very expensive order? | How big is our workforce? | How many years do we assume a packaging machine lasts? | Do we pay customs duty on control units from Tokyo? | May we buy from a blacklisted company? | What happens if a supplier is a financial risk? | How much did the business earn last year?

THIS QUESTION: KEYWORD	THIS QUESTION: MEANING	ALL 8 QUESTIONS: KEYWORD	ALL 8 QUESTIONS: MEANING
missed	found it	2 found	8 found

KEYWORD SEARCH (COUNTS SHARED WORDS)

#1 · T2 · score 3.26
Servo motors (heading 8501): 2.7 percent.

MEANING SEARCH (REAL EMBEDDINGS)

#1 · P1 · closeness 0.673 · answers the question
Vendors with a credit score of 75 or higher are low risk and receive standard Net-45 payment terms.

#2 · T2 · closeness 0.673
Servo motors (heading 8501): 2.7 percent.

#3 · P3 · closeness 0.662
Vendors scoring below 50 are high risk and require full prepayment plus a bank guarantee, or must be rejected.

Real embeddings (gemini-embedding-001) of 22 sentences from the four sample PDFs. The keyword side counts shared words and is computed live in this page. 'Answers the question' is our own judgement of which sentence is correct.

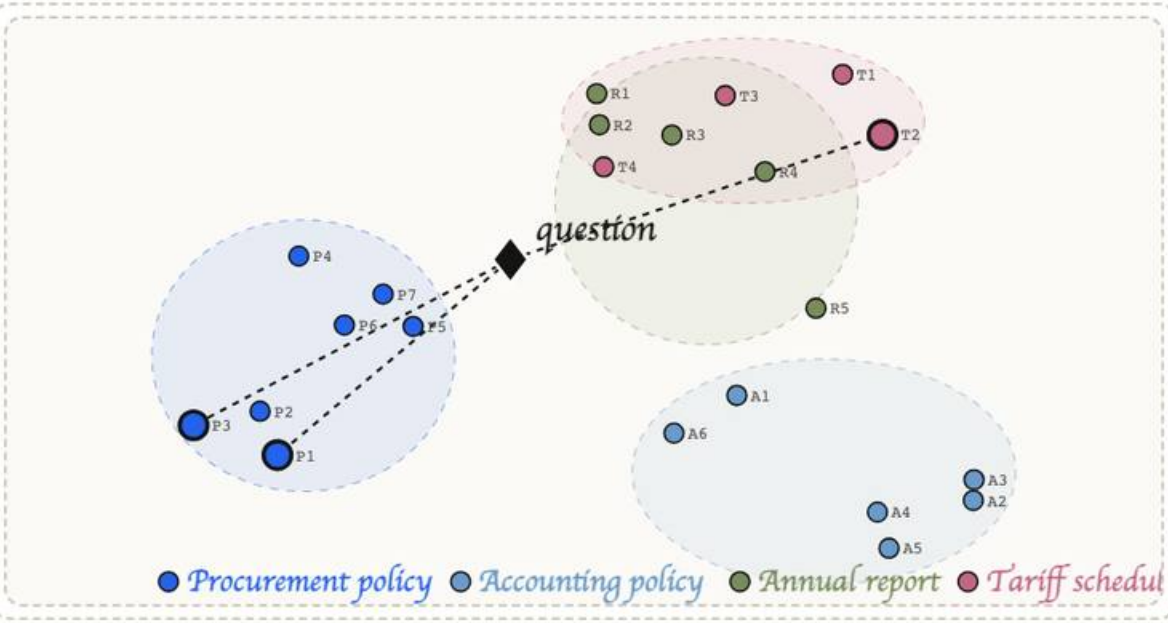
// 08 RAG EXPLAINED

The meaning map

PICK A QUESTION

When do we pay servo motor suppliers? | Who has to approve a very expensive order? | How big is our workforce? | How many years do we assume a packaging machine lasts? | Do we pay customs duty on control units from Tokyo? | May we buy from a blacklisted company? | What happens if a supplier is a financial risk? | How much did the business earn last year?

RESULTS (K) 3



QUESTION: WHEN DO WE PAY SERVO MOTOR SUPPLIERS?

#1 · P1 · closeness 0.673
Vendors with a credit score of 75 or higher are low risk and receive standard Net-45 payment terms.

#2 · T2 · closeness 0.673
Servo motors (heading 8501): 2.7 percent.

#3 · P3 · closeness 0.662
Vendors scoring below 50 are high risk and require full prepayment plus a bank guarantee, or must be rejected.

The 768 numbers of each sentence flattened to two for the picture (a projection that keeps only part of the information). The list on the right is ranked with all 768 numbers, so a result can look far away on the map.

// 08 RAG EXPLAINED

What the model is told

QUESTION **What payment terms do servo motor suppliers get at Aldermoor?**

PUT INTO THE PROMPT

- Page 1: procurement policy
- Page 2: tariff schedule (a distraction)
- Instruction: answer only from the context

THE PROMPT THE MODEL RECEIVES

```
SYSTEM:
You are a helpful assistant for question-answering tasks. Use the following
pieces of retrieved context to answer the question. If you don't know the
answer based on the context, say that you don't know. Keep the answer concise
and accurate.

Context: [local://aldermoor_procurement_policy.pdf]
Aldermoor Industries - Procurement Policy
 (fictional)
Payment terms by vendor risk
Vendors with a credit score of 75 or higher are low risk and receive standard
Net-45 payment
terms. This applies to servo motor suppliers, PLC suppliers and other
strategic component
suppliers. Vendors scoring 50 to 74 are moderate risk and require a 25 to 50
percent deposit or a
confirmed letter of credit. Vendors scoring below 50 are high risk and
require full prepayment plus a
bank guarantee, or must be rejected.
Sanctions screening
Every new vendor must be screened against EU, US OFAC, UN and German export-
control lists
```

WHAT IT ANSWERED

Correct: Net-45

Servo motor suppliers (classified as low-risk vendors with a credit score of 75 or higher) receive standard Net-45 payment terms.

Recorded run: 3.1 seconds. Model gemini-3.8-flash, temperature 0, 10 October 2026.

Eight real runs, one per combination. The instruction line is the project's own SYSTEM_PROMPT from rag/retrieval.py. A different or smaller model may behave differently, which is why the instruction matters.

// 08 RAG EXPLAINED

When RAG goes wrong

WHAT THE USER SEES

- The answer is confident but wrong
- It says 'I don't know', but the file has the answer
- The answer mixes two documents
- The answer ignores the files and uses general knowledge
- Answers got worse after changing the embedding model
- The same sentence appears twice in the answer
- Answers are slow
- The key sentence is cut in half

DIAGNOSIS

LIKELY CAUSE

The right chunk was not retrieved, or a near miss came first, so the model answered from the wrong text.

WHAT TO TRY

Look at the retrieved chunks before blaming the model. Then try a different search mode, a larger k, or a reranker.

IN MERIDIAN

rag/retrieval.py: the three strategies. The log line 'RAG query received' shows the question and the mode.

// SUMMARY

RAG in five lines

1. **A model cannot know your documents.**
   Retrieval puts the right pages in front of it.

2. **Two stages.**
   Index once (read, cut, embed, store); then, for each question, embed, search, prompt, answer.

3. **Chunks are small pieces with a little overlap.**
   Our setting is 1000 characters and 100 overlap.

4. **Embeddings turn meaning into numbers.**
   Nearby numbers mean nearby meaning, and it is not perfect.

5. **The prompt keeps the model honest.**
   Answer from the context, or say you do not know.

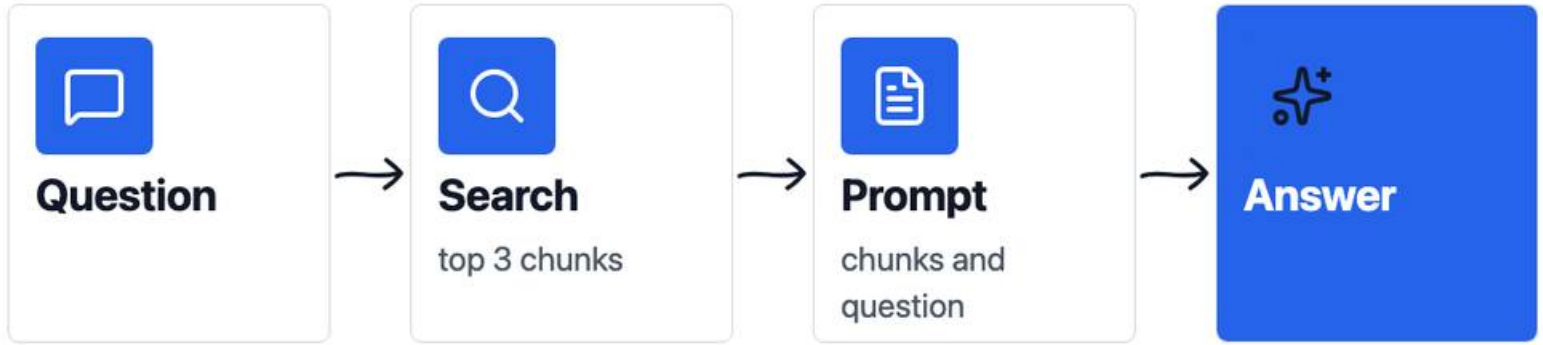
// DOCUMENT QUESTION AND ANSWER

The types of RAG, in plain words

Basic RAG, and the ways people improve it: rewrite the question, search better, clean the results



Basic RAG, and where it hurts



Different words

The question says 'expensive order', the text says 'purchases above EUR 250,000'.

Wrong order

The right chunk is found, but it is third, and a distraction is first.

Noise and gaps

A chunk may carry a lot of text that does not matter, or miss the sentence before it.

> Basic RAG works well when the question and the text share words or meaning, and the right chunk comes first.

// 09 TYPES OF RAG

Four places to improve it



Before the search

Change the question: several phrasings (multi-query), an imagined answer (HyDE), smaller questions (decomposition), a follow-up made standalone.



During the search

Change what is searched or how: keyword and meaning together (hybrid), varied results (MMR), small pieces that return their parent page.



After the search

Clean the results: reorder them (rerank), trim them (contextual compression), or check them and try again (corrective RAG).



Beyond one search

Let an agent decide what to do (agentic RAG), follow links between things (graph RAG), or read images and tables (multimodal RAG).

// 09 TYPES OF RAG

Pick a variant, see its pipeline

VARIANTS

- Basic RAG
- Multi-query RAG
- Contextual compression
- HyDE
- Query decomposition
- Follow-up rewrite
- Hybrid search
- MMR (varied results)
- Reranking
- Parent pages (small to big)
- Corrective RAG
- Agentic RAG
- Graph RAG
- Multimodal RAG

Basic RAG

In Meridian

Search with the question, put the top chunks in the prompt.

Question → Embed → Search top k → Prompt → Answer

WHAT CHANGES

Nothing: it is the baseline.

BEST FOR

Clear questions that share words or meaning with the text.

WHAT IT COSTS

One embedding call and one model call.

Orange steps are the ones that are new compared with basic RAG.

// 09 TYPES OF RAG

Multi-query RAG

what the model wrote

```
# the question
What payment terms do servo motor suppliers get?

# three rewrites by the model (real log line, 10 October 2026)
1  What are the typical payment conditions offered to servo motor suppliers?
2  Which payment terms are standard for servo motor manufacturers and vendors?
3  What kind of payment arrangements do suppliers of servo motors usually receive?

# four searches, results merged, duplicates dropped: 4 chunks
```

Why it helps

One idea can be asked in many ways. Four phrasings reach more of the stored text than one.

What it costs

One extra model call and four searches. In our run the full answer took 12 s instead of 7 s.

In Meridian

retriever_type = multiquery. We read the code in Chapter 19.

// 09 TYPES OF RAG

Contextual compression

before and after

```
# before: the whole chunk, 844 characters (shortened here)
Aldermoor Industries – Procurement Policy  (fictional) Payment terms by vendor risk
Vendors with a credit score of 75 or higher are low risk and receive standard Net-45
payment terms. This applies to servo motor suppliers, PLC suppliers and other
strategic component suppliers. Vendors scoring 50 to 74 are moderate risk and ...

# after: what the model kept, 193 characters
Vendors with a credit score of 75 or higher are low risk and receive standard Net-45
payment terms. This applies to servo motor suppliers, PLC suppliers and other
strategic component suppliers.
```

Why it helps

The prompt carries only the sentences that matter, so the model is less distracted and the prompt is shorter.

What it costs

The search fetches ten chunks and the model reads each one: ten small model calls.

In Meridian

retriever_type = contextual. Four documents went in, one trimmed chunk came out.

// 09 TYPES OF RAG

HyDE: search with an imagined answer

hypothetical document embeddings

```
# the question
When do we pay servo motor suppliers?

# the model imagines an answer (it knows nothing about Aldermoor; that is fine)
"We pay servo motor suppliers according to our agreed contract terms, typically net 30
days after receiving and inspecting the shipment."

# the search uses the imagined answer; it looks like real policy text
P1 Vendors ... receive standard Net-45 payment terms.
```

Why it helps

A short question and a long policy sentence are far apart in meaning space. An imagined answer sits closer to the real one.

The catch

The imagined answer can be wrong. It is only used to search, never shown or trusted.

Our test

Chapter 21 runs it on 22 sentences and shows what came back.

// 09 TYPES OF RAG

Split the question, or complete it

query decomposition (real run, Chapter 21)

```
# a two-part question
What payment terms do servo motor suppliers get,
and what duty do we pay on servo motors?

# split by the model
1  What payment terms do servo motor suppliers get?
2  What duty do we pay on servo motors?

# each searched separately, results merged
P1 (Net-45)   T2 (duty 2.7 percent)   T1
```

follow-up rewrite (real run, Chapter 21)

```
# a chat
Q: What terms do low risk vendors get?   A: Net-45.
Q: And for moderate risk?

# rewritten by the model so the search can understand it
What terms do moderate risk vendors get?
```

Different problems, similar trick

Both use the model to fix the question before any search happens.

// 09 TYPES OF RAG

Search better, order better

Hybrid search

Run a keyword ranking and a meaning ranking, then merge the two lists. Exact codes and names are found by the keywords; paraphrases by the meaning. In our tiny test the keyword side pushed the tariff line to first place, so hybrid is not automatically better.

Reranking

Fetch more than you need (for example eight), let a model score each one against the question, keep the best three. In our test it removed the off-topic tariff line from the top three.

MMR

Choose chunks that are relevant but unlike each other, so the prompt does not hold three copies of one idea.

Parent pages

Search with small pieces, but hand the model the whole page they came from, so the context around the match is not lost.

Results from our own run on 22 sentences, Chapter 21. They are an illustration, not a benchmark.

// 09 TYPES OF RAG

Beyond one search: four bigger ideas

Corrective RAG

A grader checks the retrieved chunks. If they are poor, the system rewrites the question or looks elsewhere before answering.

Agentic RAG

An agent decides whether to search at all, what to search for, and whether to search again. Retrieval is one tool among several.

Graph RAG

Documents become a graph of things and relations. Good for questions about connections, costly to build.

Multimodal RAG

Images, tables and slides are indexed as well, not skipped. Needs better parsing and a model that reads images.

// 09 TYPES OF RAG

Which one should I try first?

TICK WHAT YOU OBSERVE

- [ ] USERS PHRASE QUESTIONS DIFFERENTLY FROM THE DOCUMENTS
- [ ] QUESTIONS HAVE TWO OR MORE PARTS
- [ ] PEOPLE ASK FOLLOW-UP QUESTIONS IN A CHAT
- [ ] EXACT NAMES, CODES OR IDS ARE MISSED
- [ ] THE RIGHT CHUNK IS FOUND, BUT NOT FIRST
- [ ] RETRIEVED CHUNKS ARE LONG AND NOISY
- [ ] THE TOP RESULTS ARE NEAR COPIES OF EACH OTHER
- [ ] ANSWERS NEED THE TEXT AROUND THE MATCH
- [ ] THE MODEL ANSWERS EVEN WHEN THE DOCUMENTS DO NOT COVER IT
- [ ] THE TASK NEEDS SEVERAL STEPS OR TOOLS
- [ ] QUESTIONS ARE ABOUT LINKS BETWEEN THINGS
- [ ] KEY FACTS SIT IN TABLES, CHARTS OR SCANS

Clear all

TRY FIRST

Nothing ticked yet. Tick one or two symptoms on the left.

// 09 TYPES OF RAG

What Meridian uses

RETRIEVER_TYPE	VARIANT	WHAT HAPPENS	FULL ANSWER
similarity	Basic RAG	The 3 closest chunks	7.1 s
multiquery	Multi-query RAG	Model rewrites 3 ways, 4 searches, merge	12.0 s
contextual	Contextual compression	10 chunks, the model trims each	9.7 s

First run on 10 October 2026 with the four sample PDFs and gemini-3.8-flash; a second run gave 5.0, 9.8 and 5.8 s, so the numbers vary but the order held. Everything else on the explorer is shown in code in Chapter 21.

// SUMMARY

The types of RAG in five lines

1 **Basic RAG is the baseline.**
Search with the question, answer from the top chunks.

2 **Improve it in four places.**
Before the search, during it, after it, or with a bigger structure around it.

3 **Multi-query, HyDE, decomposition, follow-up rewrite**
change the question before the search.

4 **Hybrid, MMR, parent pages, reranking, compression**
change what is searched or how the results are cleaned.

5 **Start simple and measure.**
Add a variant only when you can name the symptom it cures.

// DOCUMENT QUESTION AND ANSWER

The tools behind RAG

Embedding models, vector databases and frameworks: what exists, how to choose, and where our project sits



// 10 THE RAG TOOLBOX

A RAG system has eight parts

PART	ITS JOB	TYPICAL TOOLS	IN MERIDIAN
Parse	Files to text	pypdf, PyMuPDF, Docling, Unstructured	PyPDFLoader (pypdf)
Chunk	Cut into pieces	Recursive, semantic or layout-based splitters	Recursive, 1000 and 100
Embed	Text to numbers	Gemini, OpenAI, Cohere, Voyage, BGE, Qwen3	gemini-embedding-001, 768 numbers
Store	Keep and find vectors	Chroma, FAISS, pgvector, Qdrant, Pinecone	Chroma (local), Vertex AI (cloud)
Retrieve	Choose the pieces	Similarity, multi-query, hybrid, MMR	similarity, multiquery, contextual
Rerank	Reorder (optional)	Cohere Rerank, BGE reranker, a model as judge	Not in the project (Chapter 21)
Generate	Write the answer	Gemini, GPT, Claude, open models	gemini-3.8-flash
Evaluate	Measure the answers	RAGAS, DeepEval, TruLens, your own list	The lab in Chapter 20

Names change quickly. Check each project's own documentation before you choose.

Embedding models: what to check

How many numbers
768 here. More numbers can capture more, but cost more space and time in the database.

Languages
Check that the model supports the languages of your documents and of your users.

How much text it accepts
Long chunks need a model with a long input limit.

Hosted or open
A hosted API is simple and costs per use. An open-weights model runs on your own machine.

One model for both
Index and search must use the same model, or the numbers cannot be compared.

examples, in no order of quality

```
# hosted APIs
Gemini embeddings, Vertex AI text-embedding
OpenAI text-embedding-3
Cohere Embed
Voyage AI

# open weights, run it yourself
BGE-M3
Qwen3-Embedding
Nomic Embed
E5
(Sentence Transformers runs most of them)
```

> Leaderboards such as MTEB change every few months. Test on your own documents before you choose.

Vector databases: what they do

Store
Keep each chunk's numbers, its text and its metadata (file name, page).

Find neighbours
Return the stored items closest to a question's numbers.

Filter
Search only inside one file, one year, one customer.

Keep up to date
Add, replace and delete items without rebuilding everything.

exact or approximate?

```
# exact: compare the question with every vector
simple, always right, slow when there are millions

# approximate (ANN): look at a clever subset
HNSW is the common index
much faster, a small chance of missing the best match
```

> For a few thousand chunks, either is fast. The choice starts to matter at millions.

Choose your stack

Embedding models | Vector stores | Frameworks | Rerankers | Parsers | Evaluation

SHOW ONLY: Open source / weights | Runs locally | Managed service | Hosted API | Google Cloud

8 shown

Gemini embeddings
Used in Meridian, local mode

Google's embedding models through the Gemini API (gemini-embedding-001).

GOOD FOR Projects that already use Gemini; the vector size can be chosen.

Hosted API

Vertex AI text-embedding-005
Used in Meridian, cloud mode

Google Cloud's embedding model through Vertex AI.

GOOD FOR Deployments on Google Cloud.

Hosted API | Google Cloud

OpenAI text-embedding-3
OpenAI's hosted embedding models, in a small and a large size.

GOOD FOR Teams that already use OpenAI.

Hosted API

Cohere Embed
Hosted embedding models, known for multilingual search.

GOOD FOR Documents in many languages.

Hosted API

Voyage AI
Hosted embedding models built for retrieval, with rerankers.

GOOD FOR Retrieval quality as the main goal.

Hosted API

BGE-M3
Open-weights model from BAAI: many languages and long inputs.

GOOD FOR Running embeddings on your own hardware.

Open source / weights | Runs locally

Qwen3-Embedding
Open-weights embedding models in several sizes.

GOOD FOR Self-hosting with a choice of size.

Open source / weights | Runs locally

Nomic Embed
Open embedding model with long input support.

GOOD FOR Local experiments with long chunks.

Open source / weights | Runs locally

Choosing a vector database: five questions

ASK	IT LEANS TOWARD
How many vectors will you store?	Thousands: Chroma or FAISS. Millions: Qdrant, Weaviate, pgvector or a managed service. Billions: Milvus or a managed service.
Do you already run a database or search cluster?	Postgres: pgvector. Elasticsearch or OpenSearch: use their vector search. Fewer systems to operate.
Do you want to run servers?	No: Pinecone, Vertex AI Vector Search or another managed service. Yes: Qdrant, Weaviate, Milvus, Chroma.
Do you need filters or keyword search too?	Qdrant, Weaviate, Elasticsearch and OpenSearch are strong here. Always test with your own filters.
Where must the data live?	Region, privacy and contracts can decide before performance does.

> For a few thousand chunks nearly everything works. Benchmark on your own data before you commit.

Retrieval frameworks: what they give you

What LangChain gives the project

PyPDFLoader to read, RecursiveCharacterTextSplitter to cut, one VectorStore interface over Chroma and Vertex AI Vector Search, ChatGoogleGenerativeAI, prompt templates, and the | pipe to join steps.

LlamaIndex

Strong when the whole job is indexing and querying documents.

Haystack

Pipelines made of typed components that can be tested one by one.

No framework

A provider SDK and a vector client are enough for a small app.

the same idea in the project

```
# rag/vector_store.py
Chroma(...)                    # local
VectorSearchVectorStore(...) # cloud

# both are a VectorStore, so this works with either
store.add_documents(chunks)
store.similarity_search(question, k=3)
```

A common interface is what lets the project switch from a laptop to the cloud by changing one setting.

Where Meridian sits

PART	LOCAL MODE (PART 1)	CLOUD MODE (PART 2)
Parse	PyPDFLoader	PyPDFLoader
Chunk	RecursiveCharacterTextSplitter, 1000 and 100	the same
Embed	gemini-embedding-001, 768 numbers	text-embedding-005 through Vertex AI, 768 numbers
Store	Chroma in .meridian/chroma	Vertex AI Vector Search
Originals	.meridian/uploads/ on disk	A Cloud Storage bucket
Generate	gemini-3.8-flash, Gemini API and your key	the same, key from Secret Manager

> One setting, APP_MODE, switches the whole column. The rest of the code never knows which one it got.

// SUMMARY

The RAG toolbox in five lines

1 **Eight parts, one choice each.**
Parse, chunk, embed, store, retrieve, rerank, generate, evaluate.

2 **Embedding models differ in size, languages, input length and cost.**
Use the same one to index and to search.

3 **A vector database stores vectors and finds neighbours quickly.**
Pick the simplest one that fits your size and your servers.

4 **A framework saves plumbing, not thinking.**
LangChain here; others work as well.

5 **Meridian: Gemini, Chroma, LangChain.**
Cloud mode swaps the embedding model and the store.

// BUILD THE DOCUMENT Q&A, PART ONE

Talking to Gemini, and turning text into numbers

Two small files: one shared connection to the model, and the function that turns text into 768 numbers



Put the two files in place

run from meridian-ai, then backend/ Copy

```
$ cd backend
$ cp ../../meridian-reference/backend/rag/llm.py rag/llm.py
$ cp ../../meridian-reference/backend/rag/embeddings.py rag/embeddings.py
$ wc -l rag/llm.py rag/embeddings.py
      35 rag/llm.py
      41 rag/embeddings.py
      76 total
```

> From here on every command runs from backend/. The first line moves there.

Copy them

The commands fill the two empty files from the reference copy.

Or type them

Follow the next slides and type each part.

Check the size

35 and 41 lines mean the files arrived whole.

One connection, made once

Two jobs need the model
Answering questions (retrieval.py) and the audit agents (agent/). One shared connection avoids two different configurations.

Create it on first use
Not when the file is imported. The API can start, and /api/health can answer, before any key exists.

Reuse it
Creating a model object is cheap but not free. lru_cache keeps one and hands it back every time.

Fail with a sentence
No key means a ConfigurationError that says what to do, not a stack trace.



one model, one place

who calls get_llm()

```
# rag/retrieval.py
get_llm()  # answers questions

# agent/agents.py (Module 5)
get_llm()  # the audit team
```

Gemini is called with the API key in both modes. Only the embeddings switch between the Gemini API and Vertex AI.

llm.py, part 1: get_llm

rag/llm.py, lines 8 to 25

```python
from functools import lru_cache

from langchain_google_genai import ChatGoogleGenerativeAI

from config.settings import settings
from exceptions import ConfigurationError


@lru_cache(maxsize=1)  # 1
def get_llm() -> ChatGoogleGenerativeAI:
    """Create the chat model on first use (not at import time) and reuse it afterwards."""
    if not settings.GOOGLE_API_KEY:  # 2
        raise ConfigurationError("GOOGLE_API_KEY is empty. Add it to .env
(https://aistudio.google.com/apikey).")
    return ChatGoogleGenerativeAI(  # 3
        model=settings.llm_model_name,
        google_api_key=settings.GOOGLE_API_KEY,
        temperature=settings.llm_temperature,  # 4
    )
```

1. **lru_cache**
   Remembers the first result. The second call returns the same object.

2. **A clear error**
   An empty key raises ConfigurationError with the address to get a key.

3. **The model object**
   ChatGoogleGenerativeAI, LangChain's wrapper for Gemini.

4. **From settings**
   Model name, key and temperature come from the settings object of Module 3.

llm.py, part 2: extract_text

rag/llm.py, lines 28 to 35

```python
def extract_text(content) -> str:
    """Gemini may return a list of content blocks, e.g. [{'type': 'text', 'text': '...'}]; join the text."""
    if isinstance(content, list):   # 1
        return "\n".join(
            block.get("text", "") for block in content   # 2
            if isinstance(block, dict) and block.get("type") == "text"
        )
    return content   # 3
```

1 Sometimes a list

Gemini may answer with a list of blocks instead of one string.

2 Join the text blocks

Keep every block whose type is text, and join them with a new line.

3 A plain string stays as it is

So the helper is safe to call on any reply.

Try it: ask Gemini

run from backend/ Copy

```
$ uv run python -c "from rag.llm import get_llm, extract_text; r = get_llm().invoke('Reply with exactly: hello'); print(type(r.content).__name__, repr(extract_text(r.content)))"
list 'hello'
```

> You need the key in .env and an internet connection.

list 'hello'

Gemini sent a list of blocks. extract_text turned it into the plain word.

Extra log lines

Lines starting HTTP Request or AFC come from the Google library. They are normal.

An API error?

Check the key in .env: no quotes, no spaces.

Try it: no key, clear error

run from backend/ Copy

```
$ GOOGLE_API_KEY= uv run python -c "from rag.llm import get_llm; get_llm()"
exceptions.ConfigurationError: GOOGLE_API_KEY is empty. Add it to .env
(https://aistudio.google.com/apikey).
```

> A long traceback is printed first; the last line is the message.

ConfigurationError

The error type we built in Module 3, with a message that says what to do.

An empty variable wins

A value set on the command line overrides .env, which is how we simulate a missing key.

Remember the 503

In the API this becomes HTTP 503, not a crash.

Try it: the idea with three numbers

run from backend/ Copy

```
$ uv run python ../../meridian-reference/docs/examples/05_similarity_demo.py
Question: When do we pay servo motor suppliers?

 0.98  Servo motors are paid on Net-45 terms
 0.36  Filling line equipment is depreciated over 10 years
 0.17  Revenue in FY2024 was EUR 2.84 billion
```

> No key is needed: the numbers are made up.

Three numbers per text

A toy version, invented by hand: no model, no key.

0.98 beats 0.36

The question's numbers point almost the same way as the first text.

RAG hands over the top one

The real files do exactly this with 768 numbers.

embeddings.py: the rule at the top

embeddings.py, lines 1 to 13 (the docstring)

```
"""Text -> vector conversion, used both when indexing documents and when searching.

The SAME model must be used for indexing and for querying, so the model follows APP_MODE:   1

    local  -> gemini-embedding-001 through the Gemini API (only GOOGLE_API_KEY needed)   2
    cloud  -> text-embedding-005 through Vertex AI (Google Cloud credentials)   3

Both produce 768-dimensional vectors. In cloud mode that number is fixed by the   4
Vector Search index (see .github/workflows/deploy.yml).

LangChain v1 note: `VertexAIEmbeddings` is deprecated. `GoogleGenerativeAIEmbeddings`   5
covers both the Gemini API and Vertex AI (vertexai=True), so one class serves both modes.
"""
```

1 One rule

Index and search must use the same model.

2 Local mode

gemini-embedding-001 through the Gemini API: only a key is needed.

3 Cloud mode

text-embedding-005 through Vertex AI: Google Cloud credentials.

4 768 numbers

Both produce 768 numbers. In cloud mode the index is created for that size.

5 One class, two paths

GoogleGenerativeAIEmbeddings serves both, with vertexai=True for the cloud.

embeddings.py, part 1: the switch and the cloud path

rag/embeddings.py, lines 22 to 33

```python
@lru_cache(maxsize=1)  (1)
def get_embeddings() -> GoogleGenerativeAIEmbeddings:
    if settings.is_cloud:  (2)
        if not settings.GCP_PROJECT:
            raise ConfigurationError("APP_MODE=cloud needs GCP_PROJECT_ID for Vertex AI embeddings.")
        return GoogleGenerativeAIEmbeddings(
            model=settings.embedding_model_name,
            vertexai=True,  (3)
            project=settings.GCP_PROJECT,
            location=settings.GCP_REGION,
            output_dimensionality=settings.embedding_dimensions,  (4)
        )
```

1 Create once

The same lru_cache trick as get_llm.

2 The switch

One question decides everything: are we in the cloud?

3 Vertex AI

vertexai=True plus project and region from settings.

4 The size

output_dimensionality asks for 768 numbers, taken from settings.

embeddings.py, part 2: the local path

rag/embeddings.py, lines 35 to 41

```
    if not settings.GOOGLE_API_KEY:    (1)
        raise ConfigurationError("APP_MODE=local needs GOOGLE_API_KEY in .env for embeddings.")
    return GoogleGenerativeAIEmbeddings(
        model=settings.local_embedding_model_name,    (2)
        google_api_key=settings.GOOGLE_API_KEY,
        output_dimensionality=settings.embedding_dimensions,    (3)
    )
```

1 A clear error again

No key, no embeddings, and a sentence that says so.

2 The local model

Its name comes from settings: gemini-embedding-001.

3 Same size

768 numbers, so local and cloud behave alike.

Try it: compare meanings with real numbers

backend/try_embeddings.py (create this file) Copy

```python
from rag.embeddings import get_embeddings

embeddings = get_embeddings()  # 1
question = "When do we pay servo motor suppliers?"
texts = [
    "Servo motors are paid on Net-45 terms",
    "Filling line equipment is depreciated over 10 years",
    "Revenue in FY2024 was EUR 2.84 billion",
]


def cosine(a, b):  # 4
    dot = sum(x * y for x, y in zip(a, b))
    return dot / (math.sqrt(sum(x * x for x in a)) * math.sqrt(sum(y * y for y in b)))


question_vector = embeddings.embed_query(question)  # 2
print("numbers per text:", len(question_vector))
print("first four numbers:", [round(x, 3) for x in question_vector[:4]])
print()
print("Question:", question)
for text, vector in zip(texts, embeddings.embed_documents(texts)):  # 3
    print(f"{cosine(question_vector, vector):.2f}  {text}")
```

1. **Get the embedder**
   The shared object from embeddings.py.

2. **embed_query**
   One question becomes 768 numbers.

3. **embed_documents**
   Several texts at once: this is what indexing uses.

4. **cosine**
   Closeness as one number. The slide shows it written out; libraries do the same.

Run it: real closeness

run from backend/ Copy

```
$ uv run python try_embeddings.py
numbers per text: 768
first four numbers: [-0.017, 0.003, 0.008, -0.059]

Question: When do we pay servo motor suppliers?
0.85  Servo motors are paid on Net-45 terms
0.55  Filling line equipment is depreciated over 10 years
0.57  Revenue in FY2024 was EUR 2.84 billion
```

> Scores can differ in the last digit on your machine; the order will not.

768 numbers

Every text, long or short, becomes the same number of numbers.

0.85 beats 0.55

The sentence about payment terms is clearly closest to the question.

Unrelated is not zero

0.55 and 0.57 show that real scores are squeezed into a narrow band: compare them with each other, never with zero.

The same model must index and search



Different models

Numbers from two models are like temperatures in two unknown units: they cannot be compared.



Different sizes

A store built for 768 numbers refuses 3072.



Change the model?

Index every document again with the new model, and keep one index per model.



Local and cloud

Both give 768 numbers, but they are different models. Do not point cloud queries at a local index.

// SUMMARY

Gemini and embeddings in five lines

1 **get_llm() is created once, on first use.**
lru_cache keeps one shared model.

2 **No key gives a ConfigurationError.**
A sentence, not a stack trace.

3 **extract_text joins Gemini's content blocks.**
We saw that the reply is a list.

4 **get_embeddings() follows APP_MODE.**
Gemini API locally, Vertex AI in the cloud, 768 numbers either way.

5 **Index and search use the same model.**
Always.

Storing meaning, keeping the originals

A vector database for the numbers, and a folder (or a bucket) for the original PDFs



// 12 STORES

Put the two files in place

run from backend/ Copy

```
$ cp ../../meridian-reference/backend/rag/vector_store.py rag/vector_store.py
$ cp ../../meridian-reference/backend/rag/document_store.py rag/document_store.py
$ wc -l rag/vector_store.py rag/document_store.py
      54 rag/vector_store.py
      80 rag/document_store.py
     134 total
```

> You stay in backend/ for the whole module.

Copy them

The commands fill the two empty files from the reference copy.

Or type them

Follow the next slides.

Check the sizes

54 and 80 lines, 134 in total.

// 12 STORES

Where do the numbers live?

Keep numbers and text
Each chunk is saved with its numbers, its words and where it came from.

Find neighbours
Given a question's numbers, return the closest chunks.

Survive a restart
Chroma writes to a folder, so the index is still there tomorrow.

One interface
Chroma locally, Vertex AI Vector Search in the cloud: the same three calls.

the VectorStore interface

```
# what the rest of the code uses
store.add_documents(chunks, ids=ids)
store.similarity_search(question, k=3)
store.as_retriever()

# two implementations behind it
Chroma(...)                    # local, a folder
VectorSearchVectorStore(...)   # cloud, managed
```

> Nothing else in the project knows which one it got.

vector_store.py, part 1: the local store

rag/vector_store.py, lines 19 to 27

```python
def _local_store() -> VectorStore:
    from langchain_chroma import Chroma  # imported here so cloud images do not need it loaded  1

    log.info("Using local Chroma vector store", path=settings.chroma_dir)  2
    return Chroma(
        collection_name=settings.chroma_collection,  3
        embedding_function=get_embeddings(),  4
        persist_directory=settings.chroma_dir,  5
    )
```

1 Import inside the function

Cloud images do not need Chroma loaded.

2 A log line

The log says which store was chosen.

3 A collection

A named group of chunks, like a table.

4 Who makes the numbers

The embedding function from the last chapter.

5 Where it is saved

A folder on disk: .meridian/chroma.

vector_store.py, part 2: the cloud store (a preview)

rag/vector_store.py, lines 30 to 48

```python
def _cloud_store() -> VectorStore:
    missing = settings.missing_cloud_settings()  (1)
    if missing:
        raise ConfigurationError(f"APP_MODE=cloud but these settings are empty: {', '.join(missing)}")
 (2)

    from google.cloud import aiplatform  (3)
    from langchain_google_vertexai import VectorSearchVectorStore

    log.info("Using Vertex AI Vector Search", index_id=settings.vector_search_index_id)
    aiplatform.init(project=settings.GCP_PROJECT, location=settings.GCP_REGION)
    return VectorSearchVectorStore.from_components(  (4)
        project_id=settings.GCP_PROJECT,
        region=settings.GCP_REGION,
        embedding=get_embeddings(),  # NOTE: singular "embedding", not "embeddings"  (5)
        index_id=settings.vector_search_index_id,
        endpoint_id=settings.vector_search_index_endpoint_id,
        gcs_bucket_name=settings.GCS_BUCKET_NAME,
        stream_update=True,  (6)
    )
```

1. **Check first**
   The helper from Module 3 lists the empty settings.
2. **A clear error**
   The message names every missing setting.
3. **Imports inside**
   Google Cloud packages load only in cloud mode.
4. **The cloud store**
   Same role as Chroma, built from project, region, index and endpoint.
5. **Same embedder**
   Singular name embedding: a quirk of this class.
6. **Streaming**
   New chunks become searchable quickly.

// 12 STORES

vector_store.py, part 3: connect on first use

rag/vector_store.py, lines 51 to 54

```python
@lru_cache(maxsize=1)  (1)
def get_vector_store() -> VectorStore:
    """Connect on first use, so the API can start (e.g. /api/health) before anything is configured."""
    return _cloud_store() if settings.is_cloud else _local_store()  (2)
```

1 Once

Created on the first call, reused afterwards.

2 One switch

A single expression chooses the store from APP_MODE. This is the whole switch.

// 12 STORES

Try it: store, search, clean up

backend/try_store.py (create this file) Copy

```python
store = get_vector_store()  1
store.add_texts(  2
    ["Servo motors are paid on Net-45 terms",
     "Filling line equipment is depreciated over 10 years",
     "Revenue in FY2024 was EUR 2.84 billion"],
    ids=["demo-1", "demo-2", "demo-3"],
)
for doc in store.similarity_search("When do we pay servo motor suppliers?", k=2):  3
    print(doc.page_content)
store.delete(ids=["demo-1", "demo-2", "demo-3"])  4
print("stored now:", len(store.get()["ids"]))
```

1 **Connect**
The first call creates the Chroma store.

2 **Add three texts**
With our own ids, so that we can remove them again.

3 **Search by meaning**
The two closest to the question, k=2.

4 **Clean up**
Delete the demo items so the real index stays clean.

// 12 STORES

Run it: the folder appears

run from backend/ Copy

```
$ uv run python try_store.py
$ ls ../.meridian/chroma
{"path": ".../.meridian/chroma", "timestamp": "...", "level": "info", "severity": "INFO", "event":
"Using local Chroma vector store"}
Servo motors are paid on Net-45 terms
Revenue in FY2024 was EUR 2.84 billion
stored now: 0
chroma.sqlite3
b5904786-a204-4a5d-9c51-842006552509
```

> The log line is longer on your screen. The id folder name is random.

Two results

The payment sentence is first. Revenue is second, closer than the depreciation line.

.meridian/chroma

Created in the project folder: a sqlite file and an id folder.

stored now: 0

The demo items are gone again.

// 12 STORES

Why keep the originals?

The database holds chunks
It does not hold your PDFs.

To index again you need the files
A new embedding model, a new chunk size or a bug fix means re-indexing everything.

Two operations are enough
save keeps a copy; pdfs lists the stored files.

Local or cloud
A folder .meridian/uploads, or a Cloud Storage bucket.

document_store.py in one picture

```
# local mode
.meridian/uploads/name.pdf

# cloud mode
gs://bucket/prefix/name.pdf

save(local_path, filename)
pdfs()  # every stored PDF
```

> The source name saved with each chunk shows where the file came from: local://name.pdf or gs://bucket/name.pdf.

document_store.py, part 1: save

rag/document_store.py, lines 36 to 48

```python
def save(local_path: str, filename: str) -> str:
    """Keep a copy of the uploaded PDF. Returns where it was stored."""
    filename = os.path.basename(filename)  # 1
    if settings.is_cloud:  # 2
        blob_name = f"{settings.GCS_PREFIX}{filename}"
        _bucket().blob(blob_name).upload_from_filename(local_path, content_type="application/pdf")  # 3
        stored_at = f"gs://{settings.GCS_BUCKET_NAME}/{blob_name}"
    else:
        os.makedirs(settings.local_upload_dir, exist_ok=True)
        stored_at = os.path.join(settings.local_upload_dir, filename)
        shutil.copyfile(local_path, stored_at)  # 4
    log.info("Original file stored", filename=filename, stored_at=stored_at)  # 5
    return stored_at
```

1 Only the file name

basename removes any folders: a safety measure.

2 Two places

The mode decides where the copy goes.

3 Cloud

Upload to the bucket under the prefix.

4 Local

Copy into .meridian/uploads.

5 A log line

Records where the original was stored.

document_store.py, part 2: pdfs

rag/document_store.py, lines 51 to 66

```python
@contextmanager  (1)
def pdfs() -> Iterator[Iterator[tuple[str, str]]]:
    """Yield (local_path, source) for every stored PDF.

    Used as a context manager because cloud files are downloaded to a temp folder
    that must be cleaned up afterwards.
    """
    if not settings.is_cloud:  (2)
        folder = settings.local_upload_dir
        os.makedirs(folder, exist_ok=True)
        yield (  (3)
            (os.path.join(folder, name), f"local://{name}")  (5)
            for name in sorted(os.listdir(folder))
            if name.lower().endswith(".pdf")  (4)
        )
        return
```

1 A context manager
Used with 'with': it can clean up afterwards, which the cloud branch needs.

2 Local first
In local mode, list the upload folder.

3 yield
Hands out one pair at a time: path and source name.

4 Only PDFs
Other files are skipped.

5 A source name
local://file.pdf is saved in every chunk of that file.

Try it: keep and list

backend/try_documents.py (create this file) Copy

```
print("location:", document_store.location())   1
print("saved at:",
document_store.save("../sample_docs/aldermoor_procurement_policy.pdf",
"aldermoor_procurement_policy.pdf"))   2
with document_store.pdfs() as files:   3
    for local_path, source in files:
        print("stored:", source)
```

1 **Where**
location() says the folder.

2 **Keep a copy**
save() copies the PDF into the document store.

3 **List**
pdfs() as a with block, one source name per file.

// 12 STORES

Run it: the copy is kept

run from backend/ Copy

```
$ uv run python try_documents.py
$ ls ../.meridian/uploads
{"filename": "aldermoor_procurement_policy.pdf", "stored_at":
".../.meridian/uploads/aldermoor_procurement_policy.pdf", ..., "event": "Original file stored"}
location: .../.meridian/uploads
saved at: .../.meridian/uploads/aldermoor_procurement_policy.pdf
stored: local://aldermoor_procurement_policy.pdf
aldermoor_procurement_policy.pdf
```

> Paths show your own folder.

local://

The source name that will be saved with every chunk.

The upload folder

Is shows the stored PDF.

Original file stored

A log line records every save.

// SUMMARY

The two stores in five lines

1 **A vector store keeps numbers, text and metadata.**
And finds the nearest chunks to a question.

2 **Chroma locally, Vertex AI Vector Search in the cloud.**
One VectorStore interface hides the difference.

3 **get_vector_store() connects on first use.**
So the API can start before anything is set up.

4 **The document store keeps the original PDFs.**
A folder locally, a bucket in the cloud.

5 **Originals make re-indexing possible.**
The index is a derived copy.

// BUILD THE DOCUMENT Q&A, PART THREE

Ingestion: cutting documents into pieces

Read a PDF, cut it into chunks with stable ids, turn them into numbers and store them



// 13 INGESTION

Put the file in place

run from backend/ Copy

```
$ cp ../../meridian-reference/backend/rag/data_ingestion.py rag/data_ingestion.py
$ wc -l rag/data_ingestion.py
73 rag/data_ingestion.py
```

> Windows prints just 73.

Copy it
The command fills data_ingestion.py.

Or type it
Follow the next slides.

Check the size
73 lines.

// 13 INGESTION

Why cut at all?

Too big to send
A whole document does not fit a prompt, and costs more the longer it is.

Too blurry to search
One set of numbers for a whole page describes everything and nothing.

A sentence is the answer
Small chunks match a question precisely.

SAMPLE FILE	CHARACTERS	PAGES	CHUNKS
Procurement policy	845	1	1
Accounting policy	664	1	1
Annual report FY2024	950	1	1
EU tariff excerpt	645	1	1

Honest note: each sample PDF is one short page, so it becomes exactly one chunk. The chunking code matters for your own, longer documents.

data_ingestion.py, part 1: the splitter

rag/data_ingestion.py, lines 10 to 27

```python
import hashlib

from langchain_community.document_loaders import PyPDFLoader   1
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter

from exceptions import IngestionError
from logger import GLOBAL_LOGGER as log
from rag import document_store
from rag.vector_store import get_vector_store

CHUNK_SIZE = 1000   2
CHUNK_OVERLAP = 100  # neighbouring chunks share 100 characters so sentences are not cut off
 3


def split_into_chunks(pages: list[Document]) -> list[Document]:
    splitter = RecursiveCharacterTextSplitter(chunk_size=CHUNK_SIZE, chunk_overlap=CHUNK_OVERLAP)
 4
    return splitter.split_documents(pages)   5
```

1 **The loader**
PyPDFLoader reads a PDF page by page.

2 **Size**
About 1000 characters per chunk.

3 **Overlap**
Neighbours share 100 characters, so a sentence on the boundary is not lost.

4 **The splitter**
The recursive splitter you explored in Chapter 13.

5 **Pages in, chunks out**
split_documents keeps each page's metadata on its chunks.

// 13 INGESTION

Try it: see the overlap

run from backend/ Copy

```
$ uv run python ../../meridian-reference/docs/examples/03_chunking_demo.py
485 characters -> 3 chunks (size=200, overlap=40)

--- chunk 1 (199 chars) ---
... Vendors scoring 50 to 74
--- chunk 2 (192 chars) ---
payment terms. Vendors scoring 50 to 74 are moderate risk ...
--- chunk 3 (171 chars) ---
```

> Shortened on the slide.

485 characters, 3 chunks

Size 200 and overlap 40 on a short text.

The repeated words

Chunk 2 starts with words that end chunk 1: that is the overlap.

Change the numbers

Copy the script, change CHUNK_SIZE and CHUNK_OVERLAP, run again.

data_ingestion.py, part 2: stable ids

rag/data_ingestion.py, lines 30 to 33

```python
def _chunk_id(chunk: Document, position: int) -> str:
    """Stable ID per chunk: re-indexing the same file overwrites instead of duplicating."""
    key = f"{chunk.metadata.get('source')}|{chunk.metadata.get('page')}|{position}"  (1)
    return hashlib.sha1(key.encode()).hexdigest()  (2)
```

1 A key from the facts

The source file, the page and the position of the chunk.

2 A fixed fingerprint

sha1 turns the key into a 40-character id. The same chunk always gets the same id.

// 13 INGESTION

data_ingestion.py, part 3: ingest_pdf

rag/data_ingestion.py, lines 36 to 54

```python
def ingest_pdf(local_path: str, source: str) -> dict:
    """Load one PDF, chunk it and store it. Returns {"pages": n, "chunks": n}.

    `source` is saved in each chunk's metadata so answers can be traced back to a file.
    """
    try:
        pages = PyPDFLoader(local_path).load()  # 1
    except Exception as exc:  # pypdf raises many different error types
        raise IngestionError(f"Could not read PDF '{source}': {exc}") from exc  # 2

    for page in pages:
        page.metadata["source"] = source  # 3

    chunks = split_into_chunks(pages)  # 4
    if chunks:
        log.info("Embedding chunks and adding to vector store", source=source, chunks=len(chunks))
        ids = [_chunk_id(c, i) for i, c in enumerate(chunks)]  # 5
        get_vector_store().add_documents(chunks, ids=ids)  # 6
    return {"pages": len(pages), "chunks": len(chunks)}
```

1. **Read**
   One Document per page.
2. **Our own error**
   A bad PDF becomes an IngestionError (HTTP 422).
3. **Remember the source**
   Every page gets the file name.
4. **Cut**
   Pages become chunks.
5. **Ids**
   One stable id per chunk.
6. **Embed and store**
   Same id means replace.

data_ingestion.py, part 4: ingest_stored_pdfs

rag/data_ingestion.py, lines 57 to 73

```python
def ingest_stored_pdfs() -> dict:
    """Index every PDF already in the document store."""
    log.info("Indexing stored documents", location=document_store.location())  1
    totals = {"files": 0, "pages": 0, "chunks": 0}

    with document_store.pdfs() as files:  2
        for local_path, source in files:
            result = ingest_pdf(local_path, source=source)  3
            log.info("Ingested stored PDF", source=source, **result)
            totals["files"] += 1  4
            totals["pages"] += result["pages"]
            totals["chunks"] += result["chunks"]

    if totals["files"] == 0:
        log.warning("No PDF documents found in the document store",
location=document_store.location())  5
    log.info("Ingestion complete", **totals)
    return totals
```

1. **Say where**
   A log line names the document store.

2. **Every stored PDF**
   The with block from the document store.

3. **Same function**
   ingest_pdf does the work for each file.

4. **Totals**
   Files, pages and chunks are counted.

5. **A warning, not a crash**
   An empty store is logged, not an error.

// 13 INGESTION

Try it: index twice, count once

backend/try_ingest.py (create this file) Copy

```python
from rag.vector_store import get_vector_store

PDF = "../sample_docs/aldermoor_procurement_policy.pdf"
SOURCE = "local://aldermoor_procurement_policy.pdf"

print("first run :", ingest_pdf(PDF, SOURCE))   1
print("stored    :", len(get_vector_store().get()["ids"]))   2
print("second run:", ingest_pdf(PDF, SOURCE))   3
print("stored    :", len(get_vector_store().get()["ids"]))
```

1 **One PDF**
The procurement policy, with the source name the document store would give it.

2 **Count**
get() returns every stored id; we count them.

3 **Again**
Index the same file a second time and count again.

Run it: the count stays at one

run from backend/ Copy

```
$ uv run python try_ingest.py
first run : {'pages': 1, 'chunks': 1}
stored    : 1
second run: {'pages': 1, 'chunks': 1}
stored    : 1
```

> Log lines (JSON) appear between these lines on your screen.

1 page, 1 chunk

The sample PDF is one short page.

Second run, still 1

The id was the same, so the chunk was replaced.

Random ids?

Two chunks after the second run, and duplicate answers.

// 13 INGESTION

Index all four sample PDFs

run from backend/ Copy

```
$ mkdir -p ../.meridian/uploads
$ cp ../sample_docs/*.pdf ../.meridian/uploads/
$ uv run python -c "from rag.data_ingestion import ingest_stored_pdfs; print(ingest_stored_pdfs())"
(log lines: Indexing stored documents, then one Embedding... and one Ingested stored PDF per file)
{'files': 4, 'pages': 4, 'chunks': 4}
```

> Windows needs different commands for the first two lines; see the reference.

Into the store first

The PDFs are copied into .meridian/uploads, the document store.

Then indexed

ingest_stored_pdfs reads every stored PDF and indexes it.

4, 4, 4

Four files, four pages, four chunks.

// SUMMARY

Ingestion in five lines

1. **A PDF becomes pages, pages become chunks.**
   About 1000 characters, 100 overlap.

2. **Each chunk keeps its source.**
   So an answer can be traced to a file.

3. **Ids come from file, page and position.**
   Indexing again replaces instead of duplicating.

4. **A bad PDF raises IngestionError.**
   A clear 422, not a crash.

5. **Two entry points share the steps.**
   ingest_pdf for one file, ingest_stored_pdfs for all.

// BUILD THE DOCUMENT Q&A, PART FOUR

Retrieval and the answer

Find the right chunks three ways, build the prompt, and let Gemini answer from them



// 14 RETRIEVAL

Put the file in place

run from backend/ Copy

```
$ cp ../../meridian-reference/backend/rag/retrieval.py rag/retrieval.py
$ wc -l rag/retrieval.py
115 rag/retrieval.py
```

> Windows prints just 115.

Copy it
The command fills retrieval.py.

Or type it
Follow the next slides.

Check the size
115 lines.

// 14 RETRIEVAL

What retrieval must do

Find
Return the chunks closest in meaning. Three strategies, one signature: question in, documents out.

Format
Turn the chunks into one block of text with their source names.

Instruct
Tell the model to answer only from that text, and to say so if it cannot.

Answer
Call Gemini and return plain text.

question in, answer out

```
# one function the API calls
ask_question(query, retriever_type="similarity")

# three strategies, same shape
_similarity(question)  -> list[Document]
_multiquery(question)  -> list[Document]
_contextual(question)  -> list[Document]
```

> Because all three have the same shape, the chain does not care which one it is given.

retrieval.py, part 1: the prompts

rag/retrieval.py, lines 21 to 36

```
SYSTEM_PROMPT = (  1
    "You are a helpful assistant for question-answering tasks. "
    "Use the following pieces of retrieved context to answer the question. "
    "If you don't know the answer based on the context, say that you don't know. "  2
    "Keep the answer concise and accurate."
    "\n\n"
    "Context: {context}"  3
)

ANSWER_PROMPT = ChatPromptTemplate.from_messages([("system", SYSTEM_PROMPT), ("human", "{input}")])
 4

EXTRACT_PROMPT = ChatPromptTemplate.from_template(  5
    "Copy, word for word, only the sentences from the context that help answer the question. "
    "If nothing is relevant, reply exactly NO_OUTPUT.\n\n"  6
    "Question: {question}\n\nContext:\n{context}"
)
```

1. **The job description**
   Answer from the retrieved context.
2. **The honest exit**
   Say you do not know: the line from Chapter 13.
3. **A slot**
   {context} is filled with the chunks.
4. **The answer prompt**
   System text plus the question.
5. **A second prompt**
   Used only by contextual compression.
6. **A marker word**
   NO_OUTPUT: nothing relevant here.

retrieval.py, part 2: the plain search

rag/retrieval.py, lines 39 to 51

```python
class AlternativeQuestions(BaseModel):  # 1
    """Different phrasings of the user's question, used to widen the search."""

    questions: list[str] = Field(description="3 alternative phrasings of the question")  # 2


# ---------------------------------------------------------------------------
# Retrieval strategies: each one is a function  question -> list[Document]
# ---------------------------------------------------------------------------

def _similarity(question: str, k: int = 3) -> list[Document]:  # 3
    """The k chunks closest in meaning to the question (fast, default)."""
    return get_vector_store().similarity_search(question, k=k)  # 4
```

1 A shape for rewrites

A small model of the data we want back: a list of questions.

2 A description

It tells Gemini what the list should hold.

3 The default strategy

A function from question to documents.

4 One line

Ask the vector store for the k closest chunks. k is 3.

retrieval.py, part 3: multi-query

rag/retrieval.py, lines 54 to 67

```python
def _multiquery(question: str) -> list[Document]:
    """Gemini rewrites the question 3 ways; we search with all 4 and merge the results."""
    rewriter = get_llm().with_structured_output(AlternativeQuestions)  # 1
    rewrites = rewriter.invoke(f"Write 3 alternative phrasings of this question: {question}")  # 2
    queries = [question, *rewrites.questions]  # 3
    log.info("MultiQuery rewrites", queries=queries)

    seen, merged = set(), []  # 4
    for query in queries:
        for doc in _similarity(query):
            if doc.page_content not in seen:  # drop duplicates found by several queries  # 5
                seen.add(doc.page_content)
                merged.append(doc)
    return merged  # 6
```

1 Structured output

Gemini must answer in the AlternativeQuestions shape: a clean list.

2 Three rewrites

One model call.

3 Four queries

The original plus the three rewrites.

4 Remember what we saw

A set of texts already found.

5 Skip duplicates

A chunk found by two queries is kept once.

6 Merged

Up to four chunks, in the order found.

// 14 RETRIEVAL

retrieval.py, part 4: contextual compression

rag/retrieval.py, lines 70 to 79

```python
def _contextual(question: str) -> list[Document]:
    """Fetch 10 chunks, then Gemini trims each one to the relevant sentences (drops the rest)."""
    docs = _similarity(question, k=10)  (1)
    extractor = EXTRACT_PROMPT | get_llm() | StrOutputParser()  (2)
    extracts = extractor.batch([{"question": question, "context": d.page_content} for d in docs])  (3)
    return [
        Document(page_content=text.strip(), metadata=doc.metadata)
        for doc, text in zip(docs, extracts)
        if text.strip() and "NO_OUTPUT" not in text  (4)
    ]
```

1 Fetch more

Ten chunks instead of three.

2 A tiny chain

Prompt, model, text: the same pipe idea as the main chain.

3 batch

Runs the model on every chunk, in parallel.

4 Keep the useful ones

Empty answers and NO_OUTPUT are dropped. What stays is only the relevant sentences.

retrieval.py, part 5: choosing a strategy

rag/retrieval.py, lines 82 to 93

```python
STRATEGIES = {"similarity": _similarity, "multiquery": _multiquery, "contextual": _contextual}  # 1


def build_retriever(retriever_type: str = "similarity") -> RunnableLambda:  # 2
    if retriever_type not in STRATEGIES:
        log.warning("Unknown retriever_type, using similarity", retriever_type=retriever_type)
        retriever_type = "similarity"
    return RunnableLambda(STRATEGIES[retriever_type], name=f"{retriever_type}_retriever")  # 3


def format_docs(docs: list[Document]) -> str:  # 4
    return "\n\n".join(f"[{d.metadata.get('source', '?')}]\n{d.page_content}" for d in docs)
```

1 A dictionary of functions

The name from the API request picks the function.

2 A safe default

An unknown name logs a warning and falls back to similarity.

3 A pipeline piece

RunnableLambda wraps a plain function so that it can be joined with the pipe.

4 Format the chunks

Source name and text, separated by blank lines.

retrieval.py, part 6: the chain

rag/retrieval.py, lines 100 to 115

```python
def build_rag_chain(retriever_type: str = "similarity"):  1
    """question (str) -> answer (str), written as one LCEL pipeline."""
    return (
        {"context": build_retriever(retriever_type) | format_docs, "input": RunnablePassthrough()}
 2
        | ANSWER_PROMPT  3
        | get_llm()  4
        | StrOutputParser()  5
    )


def ask_question(query: str, retriever_type: str = "similarity") -> str:  6
    """Answer a question using the documents stored in the vector store."""
    log.info("RAG query received", query=query, retriever_type=retriever_type)
    answer = build_rag_chain(retriever_type).invoke(query)
    log.info("RAG answer generated", answer=answer)
    return answer
```

1. **Question in**
   A plain string goes in.
2. **Two inputs at once**
   The context is retrieved and formatted; the question passes through unchanged.
3. **Fill the prompt**
   Both inputs go into the answer prompt.
4. **Call Gemini**
   The shared model from Chapter 16.
5. **Plain text out**
   The reply becomes a string.
6. **The entry point**
   Log the question, run the chain, log the answer.

// 14 RETRIEVAL

The chain, pipe by pipe



the same thing as text

```
{"context": retriever | format_docs, "input": RunnablePassthrough()}  |  ANSWER_PROMPT  |  get_llm()  |  StrOutputParser()
```

> Read the pipe | as "and then". Change the first piece and you change the search mode; nothing else moves.

Try it: three modes, one question

backend/try_modes.py (create this file) Copy

```python
from rag.retrieval import ask_question, build_retriever

QUESTION = "What payment terms do servo motor suppliers get?"

for mode in ("similarity", "multiquery", "contextual"):   1
    chunks = build_retriever(mode).invoke(QUESTION)   2
    print(f"\n=== {mode}: {len(chunks)} chunk(s) go into the prompt")
    for chunk in chunks:
        print(f"  {chunk.metadata['source']}  ({len(chunk.page_content)} characters)")

    started = time.time()
    answer = ask_question(QUESTION, mode)   3
    print(f"  one full answer took {time.time() - started:.1f} s")
    print("  " + answer.replace("\n", "\n  "))
```

1 **Each mode**
The same question, three strategies.

2 **Look at the chunks**
How many go into the prompt, and from which files.

3 **Time the answer**
A full answer, from question to text.

Run it: compare the modes

run from backend/ Copy

```
$ uv run python try_modes.py
=== similarity: 3 chunk(s) go into the prompt
  one full answer took 7.1 s
  Servo motor suppliers ... receive standard Net-45 payment terms.
=== multiquery: 4 chunk(s) go into the prompt
  one full answer took 12.0 s
=== contextual: 1 chunk(s) go into the prompt
  local://aldermoor_procurement_policy.pdf  (193 characters)
  one full answer took 9.7 s
```

> Shortened on the slide. Times vary with your connection and the model.

Similarity: 3 chunks

The closest three, whatever they say. The procurement policy is first.

Multi-query: 4 chunks

Four phrasings reached all four documents.

Contextual: 1 chunk, 193 characters

Ten chunks fetched, one trimmed sentence pair kept.

// 14 RETRIEVAL

Which mode when?

RETRIEVER_TYPE	VARIANT	MODEL CALLS	WHAT GOES IN THE PROMPT	OUR RUN
similarity	Basic RAG	1 (the answer)	3 chunks	7.1 s
multiquery	Multi-query	2 (rewrite, answer)	up to 4 chunks, merged	12.0 s
contextual	Contextual compression	1 + one per chunk fetched	only the relevant sentences	9.7 s

Start with similarity
Fast and cheap. Right for clear questions.

Multi-query for vague questions
Or when documents use other words.

Contextual for long, noisy chunks
When a short, focused prompt matters.

First run on 10 October 2026, four sample PDFs, gemini-3.8-flash. A second run gave 5.0, 9.8 and 5.8 s: the order held, the numbers did not.

// SUMMARY

Retrieval in five lines

1. **Three strategies, one shape.**
   Question in, documents out.

2. **A dictionary picks the strategy.**
   retriever_type from the API request.

3. **The prompt keeps the model honest.**
   Answer from the context, or say you do not know.

4. **One chain, joined with pipes.**
   Retriever, prompt, model, parser.

5. **Choose by cost.**
   Similarity first; multi-query or contextual when you can name the symptom.

Lab: ask the four documents

Five questions, five expected answers, and the log that proves what the system did



// 15 LAB

The plan: five questions, five answers

QUESTION	EXPECTED ANSWER	WHERE IT LIVES
Total revenue in FY2024?	EUR 2.84 billion	Annual report
Payment terms for servo motor suppliers?	Net-45	Procurement policy
Customs duty on programmable controllers from Japan?	2.2 percent	Tariff excerpt
Over how many years is filling line equipment depreciated?	10 years, straight-line	Accounting policy
Who won the football World Cup in 2022?	I don't know (not in the documents)	None: the honest answer

> The last question is deliberate: the right answer is an honest refusal.

// 15 LAB

The lab script

backend/try_questions.py (create this file) Copy

```
QUESTIONS = [   1
    "What was the total revenue in FY2024?",
    "What payment terms do servo motor suppliers get?",
    "What is the customs duty on programmable controllers imported from Japan?",
    "Over how many years is filling line equipment depreciated?",
    "Who won the football World Cup in 2022?",   2
]

for number, question in enumerate(QUESTIONS, 1):
    print(f"\n{number}. {question}")
    print("   " + ask_question(question).replace("\n", "\n   "))   3
```

1 Five questions
Four from the documents, one that is not.

2 The football question
Not in any document: the model must refuse.

3 Default mode
ask_question with no mode uses similarity.

// 15 LAB

Run it: all five

run from backend/ Copy

```
$ uv run python try_questions.py
1. What was the total revenue in FY2024?  ->  EUR 2.84 billion
2. Payment terms for servo motor suppliers  ->  Net-45
3. Customs duty on controllers from Japan  ->  2.2 percent
4. Filling line equipment  ->  10 years, straight-line
5. Who won the World Cup in 2022?  ->  an honest 'I do not know' (the wording varies)
```

> The slide shortens the answers.

Four facts

EUR 2.84 billion, Net-45, 2.2 percent, 10 years.

One refusal

An honest 'I do not know': the wording varies, the refusal does not.

About 24 seconds

Five answers, one model call each.

// 15 LAB

Read the log

run from backend/ Copy

```
$ grep -h "RAG query received" logs/*.log | tail -n 2
{"query": "Over how many years is filling line equipment depreciated?", "retriever_type":
"similarity", ..., "event": "RAG query received"}
{"query": "Who won the football World Cup in 2022?", "retriever_type": "similarity", ...,
"event": "RAG query received"}
```

> Every question is one line, with its mode.

One line per question

With the question and the mode.

And the answer

The line 'RAG answer generated' holds the text.

At 2 a.m.

This is how you find out what the system said, and why.

// 15 LAB

If something goes wrong

WHAT THE USER SEES

- A fact is wrong
- 'I don't know' for a question the files answer
- A 503 or an API error
- The football question gets a real answer
- Answers are slow

DIAGNOSIS

LIKELY CAUSE

A different chunk was retrieved, or the model mixed two chunks.

WHAT TO TRY

Check the log for the question and mode, then look at the chunks that were found (try_modes.py). Try multiquery.

IN MERIDIAN

rag/retrieval.py

// 15 LAB

Commit the document Q&A

run from backend/, then meridian-ai

Copy

```
$ cd ..
$ git add -A
$ git commit -m "RAG: document question and answer"
$ git status --short
[main d724506] RAG: document question and answer
 12 files changed, 488 insertions(+)
(then git status prints nothing)
```

> The try_*.py scripts are committed too: they are your lab notes.

One safe point

The whole RAG section is saved.

Key and index stay out

.env, .meridian/ and logs/ are ignored.

Your lab notes

The try_*.py scripts are committed too.

// SUMMARY

Document Q&A in five lines

1. **RAG = look it up, then answer.**
   Retrieval first, generation second.

2. **Index once, ask many times.**
   Read, cut, embed, store; then embed, search, prompt, answer.

3. **Everything is a small file.**
   llm, embeddings, vector_store, document_store, data_ingestion, retrieval.

4. **The prompt keeps it honest.**
   The football question proves it.

5. **The log tells you what happened.**
   Question, mode, answer: one line each.

// EXTENSION: THE VARIANTS FROM CHAPTER 14, IN CODE

More RAG in code

MMR, HyDE, hybrid search, query decomposition, follow-up rewrite, reranking and parent pages, each in a few lines



The lab: 22 sentences, one question

the question

```
When do we pay servo motor suppliers?
```

ID	SENTENCE	ROLE
P1	Vendors with a credit score of 75 or higher ... receive standard Net-45 payment terms.	The answer
T2	Servo motors (heading 8501): 2.7 percent.	A tempting distraction: it mentions servo motors
P3	Vendors scoring below 50 are high risk and require full prepayment ...	Same page, wrong risk class
P2	Vendors scoring 50 to 74 are moderate risk ... deposit or letter of credit.	Same page, wrong risk class

> The 22 sentences are copied from the four sample PDFs, with ids: P (procurement), A (accounting), R (annual report), T (tariff). They are the same sentences as the interactive slides in Chapter 13.

// 16 MORE RAG IN CODE

First file: the sentences

backend/variants_data.py (create this file; Copy puts all 22 sentences on the clipboard) Copy

```
SENTENCES = [
    ("P1", "aldermoor_procurement_policy.pdf", "Vendors with a credit score of 75 or higher are low risk and receive standard Net-45 payment terms."),
    ("P2", "aldermoor_procurement_policy.pdf", "Vendors scoring 50 to 74 are moderate risk and require a 25 to 50 percent deposit or a confirmed letter of credit."),
    ("P3", "aldermoor_procurement_policy.pdf", "Vendors scoring below 50 are high risk and require full prepayment plus a bank guarantee, or must be rejected."),
    ("P4", "aldermoor_procurement_policy.pdf", "Every new vendor must be screened against EU, US OFAC, UN and German export-control lists before a purchase order is
released."),
    ("P5", "aldermoor_procurement_policy.pdf", "A sanctions match means immediate rejection."),
    # ... 22 sentences in all: P1 to P7, A1 to A6, R1 to R5, T1 to T4
]
```

Data, not logic

A list of (id, file, sentence). Nothing to explain.

Ids

P, A, R and T tell us which document each sentence came from.

Same as before

The sentences of the click-through slides.

variants_lab.py: setup and the baseline

backend/variants_lab.py, lines 14 to 23 (the Copy button copies the whole file) Copy

```python
DOCS = [Document(page_content=text, metadata={"id": id_, "source": source}) for id_, source, text in SENTENCES]  1
BY_ID = {doc.metadata["id"]: doc for doc in DOCS}  2

store = Chroma(collection_name="variants_lab", embedding_function=get_embeddings(), persist_directory=settings.chroma_dir)  3
store.reset_collection()  # start clean every run  4
store.add_documents(DOCS, ids=list(BY_ID))  5


def basic(question, k=3):  6
    return store.similarity_search(question, k=k)
```

1 One Document per sentence
With its id and file in the metadata.

2 A lookup table
From id back to document.

3 A separate collection
variants_lab, in the same Chroma folder. The main index is untouched.

4 Start clean
Every run rebuilds the collection.

5 Index the sentences
Using ids so a rerun replaces.

6 The baseline
The plain similarity search: Chapter 19's one line.

// 16 MORE RAG IN CODE

MMR and HyDE

backend/variants_lab.py, lines 26 to 35

```python
def mmr(question, k=3):
    """Maximal marginal relevance: relevant, but not three copies of the same idea."""
    return store.max_marginal_relevance_search(question, k=k, fetch_k=10)  (1)


def hyde(question, k=3):
    """Search with an imagined answer instead of the question (HyDE)."""
    reply = get_llm().invoke(f"Write one sentence that could answer this question: {question}")  (2)
    imagined = extract_text(reply.content)  # a plain string; the embedding client rejects reply.text  (3)
    return store.similarity_search(imagined, k=k)  (4)
```

1 MMR

One line: fetch ten candidates, then pick three that are relevant and different from each other.

2 An imagined answer

The model writes one sentence that could answer the question.

3 A plain string

extract_text, not reply.text: Google's embedding client rejects the special string type that .text returns. We found this by running it.

4 Search with it

The imagined sentence is embedded and searched, instead of the question.

// 16 MORE RAG IN CODE

Hybrid search, part 1: the keyword ranking

backend/variants_lab.py, lines 38 to 50

```python
def words(text):  # 1
    return set(re.findall(r"[a-z0-9]+", text.lower()))


def keyword_rank(question):
    """A tiny keyword ranking: count the shared words, and let rare words count more."""
    vocabulary = set().union(*(words(d.page_content) for d in DOCS))  # 2
    rarity = {w: math.log(len(DOCS) / sum(w in words(d.page_content) for d in DOCS)) for w in vocabulary}  # 3
    score = {}
    for doc in DOCS:
        shared = words(question) & words(doc.page_content)  # 4
        score[doc.metadata["id"]] = sum(rarity[w] for w in shared)
    return sorted(score, key=score.get, reverse=True)  # 5
```

1 Words of a text

Lower-case letters and numbers.

2 All words

The vocabulary of the 22 sentences.

3 Rare words weigh more

A word found in few sentences scores higher (inverse document frequency).

4 Shared words

Count the words the question and the sentence share, weighted.

5 A ranking

All ids, best first.

Hybrid search, part 2: fuse the two rankings

backend/variants_lab.py, lines 53 to 60

```
def hybrid(question, k=3):
    """Keyword ranking plus meaning ranking, merged with reciprocal rank fusion."""
    by_meaning = [d.metadata["id"] for d in store.similarity_search(question, k=len(DOCS))]   1
    fused = {}
    for ranking in (keyword_rank(question), by_meaning):   2
        for position, id_ in enumerate(ranking):
            fused[id_] = fused.get(id_, 0) + 1 / (60 + position)   3
    return [BY_ID[id_] for id_ in sorted(fused, key=fused.get, reverse=True)[:k]]   4
```

1 The meaning ranking

Every sentence, closest first.

2 Two rankings

The keyword one and the meaning one.

3 Reciprocal rank fusion

A sentence scores more for a high position in either list. 60 is the usual damping constant.

4 The best k

Highest fused score first.

Query decomposition

backend/variants_lab.py, lines 63 to 75

```python
class SubQuestions(BaseModel):   # 1
    questions: list[str] = Field(description="the separate simple questions inside the question, at most 3")


def decompose(question, k=2):
    """Split a many-part question into simple ones, search for each, merge the results."""
    splitter = get_llm().with_structured_output(SubQuestions)   # 2
    parts = splitter.invoke(f"Split this into separate simple questions: {question}")
    found = {}
    for sub in parts.questions:   # 3
        for doc in store.similarity_search(sub, k=k):
            found[doc.metadata["id"]] = doc   # 4
    return list(found.values()), parts.questions   # 5
```

1 A shape

A list of simple questions.

2 Split

The model returns a clean list of sub-questions.

3 One search each

Two chunks per sub-question.

4 Merge by id

A chunk found twice is kept once.

5 Both

The documents and the sub-questions, so that we can print them.

// 16 MORE RAG IN CODE

Follow-up rewrite

backend/variants_lab.py, lines 78 to 83

```python
def standalone(history, follow_up):  # 1
    """Turn a follow-up such as 'And for moderate risk?' into a question that makes sense alone."""
    reply = get_llm().invoke(
        f"Chat so far: {history}\nRewrite the last question so it can be understood alone: {follow_up}"  # 2
    )
    return extract_text(reply.content)  # 3
```

1 Two inputs

The chat so far and the new, incomplete question.

2 One instruction

Rewrite the last question so that it can be understood alone.

3 A plain string

Ready for the search.

// 16 MORE RAG IN CODE

Reranking with the model as judge

backend/variants_lab.py, lines 86 to 98

```python
class Scores(BaseModel):  # 1
    scores: list[int] = Field(description="one relevance score from 0 to 10 per sentence, in the same order")


def rerank(question, k=3):
    """Fetch 8 candidates, let the model score each one, keep the best k."""
    candidates = store.similarity_search(question, k=8)  # 2
    numbered = "\n".join(f"{n}. {d.page_content}" for n, d in enumerate(candidates, 1))  # 3
    result = get_llm().with_structured_output(Scores).invoke(  # 4
        f"Question: {question}\nScore each sentence from 0 to 10 for how well it answers the question.\n{numbered}"
    )
    ranked = sorted(zip(result.scores, candidates), key=lambda pair: pair[0], reverse=True)  # 5
    return [doc for _, doc in ranked[:k]]  # 6
```

1 A list of scores
One number from 0 to 10 per sentence.

2 Over-fetch
Eight candidates instead of three.

3 Number them
So that the scores can be matched back.

4 Ask the judge
One call scores all eight against the question.

5 Sort
Best score first.

6 Keep three
The top k after reranking.

Parent pages: small to big

backend/variants_lab.py, lines 101 to 108

```python
def parent_page(question, k=3):
    """Search small pieces, hand over the whole page they came from."""
    pages = {}  (1)
    for hit in store.similarity_search(question, k=k):
        source = hit.metadata["source"]  (2)
        text = " ".join(d.page_content for d in DOCS if d.metadata["source"] == source)  (3)
        pages[source] = Document(page_content=text, metadata={"id": source})
    return list(pages.values())  (4)
```

1 A dictionary of pages

One entry per page, so a page found twice appears once.

2 Which page?

Each sentence knows its file.

3 The whole page

All sentences of that file, joined.

4 Pages, not sentences

The model receives the context around the match.

Run the lab: every method

run from backend/ Copy

```
$ uv run python variants_lab.py
basic        ['P1', 'T2', 'P3']
mmr          ['P1', 'T2', 'T4']
hyde         ['P1', 'P4', 'T2']
hybrid       ['T2', 'P1', 'P2']
rerank       ['P1', 'P3', 'P2']
parent_page  ['aldermoor_procurement_policy.pdf', 'eu_tariff_schedule_excerpt.pdf']
decompose    ['T2', 'P1', 'T1'] ['What payment terms do servo motor suppliers get?',
'What duty do we pay on servo motors?']
standalone   What terms do moderate risk vendors get?
```

> Basic, MMR, hybrid and parent pages repeat exactly. The model-based lines may differ.

Seven methods

Each prints the ids of what it found.

About 25 seconds

Four methods call the model.

Not identical every time

HyDE, reranking, decomposition and the rewrite use the model, so they can differ a little between runs.

What came back, honestly (1 of 2)

METHOD	WHAT IT FOUND	WHAT IT SHOWS
basic	P1, T2, P3	The answer is first, but T2 (the servo duty) and P3 (the wrong risk class) come along.
MMR	P1, T2, T4	More variety, but here variety means one more unrelated sentence.
HyDE	P1, P4, T2 (another run: P1, P2, P3)	The imagined answer was about payment, so procurement sentences rise. It changes between runs.
hybrid	T2, P1, P2	The word 'servo' in the keyword ranking pushed the tariff line to first place.

A single question on 22 sentences: an illustration, not a benchmark. Run on 10 October 2026 with gemini-3.8-flash and gemini-embedding-001.

// 16 MORE RAG IN CODE

What came back, honestly (2 of 2)

METHOD	WHAT IT FOUND	WHAT IT SHOWS
rerank	P1, P3, P2	The judge removed T2 from the top three: the best result of the lab.
parent pages	procurement page, tariff page	Whole pages: the answer with its context, in a larger prompt.
decompose	T2, P1, T1	Both parts of a two-part question were found: the duty (T2) and the terms (P1).
follow-up	What terms do moderate risk vendors get?	The chat fragment became a question that can be searched.

A single question on 22 sentences: an illustration, not a benchmark. Run on 10 October 2026 with gemini-3.8-flash and gemini-embedding-001.

What to take away

No variant wins everywhere

Hybrid made this question worse and reranking made it better. On another question it could be the other way round.

Each one has a price

Extra model calls, a second index, bigger prompts. Pay only for what cures a symptom.

Measure on your questions

Keep a list of real questions with known answers, as in the lab, and compare variants on it.

Start simple

Basic RAG, then one variant at a time, then measure again.

// SUMMARY

More RAG in five lines

1. **MMR and parent pages change what is returned.**
   Variety and context, at no model cost.

2. **HyDE, decomposition and the follow-up rewrite fix the question.**
   One extra model call each.

3. **Hybrid fuses keyword and meaning rankings.**
   It helps with exact terms, not always.

4. **Reranking reorders the candidates.**
   It removed the distraction in our test.

5. **Measure, then choose.**
   No variant is free and none wins everywhere.